SIMATS ENGINEERING

Department of Information Technology

CSA3711 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Student Name	P.Ranjithkumar	**Register Number**	192411119
Department / Section	BE.CSE	**Academic Year / Semester**	2024
Faculty Name	DR.Kesavan R	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	1.11.2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5	**5**	All four engineering features (planning, measurement, standards, repeatability) judged against case evidence and mapped to incidents; engineered approach clearly stated.
2	Evaluate the Role of Process in Software Quality	2	**2**	Clear process-over-people position; Table 2 tests each incident; 38/100 coding-defect argument and TMM Level 1 placement well justified.
3	Evaluate Testing as a Process	6	**6**	Full test process, V&V per phase, detection phase per incident; cost ₹26,10,500 computed correctly; defect-flow model and limits explicitly stated.
4	Apply and Critique Basic Definitions	4	**4**	Definitions applied to MediQueue; error/fault/failure for I3, I5, I6 correct; I8 linked to a wrong oracle; glossary contents justified.
5	Evaluate the Application of Testing Principles	3	**3**	Six principles each tied to an incident or decision; “testing shows presence, not absence” used well; strongest principle defended with cost data.
6	Evaluate the Tester’s Role	3	**3**	Clear position; tester role per phase, pros/cons of self-testing, 7/2/1 catch table and healthcare-specific tester qualities.
7	Trace the Origins of Defects	4	**4**	Origin and evidence for all ten incidents with second-origin reasoning; pattern (50% requirements/design) matches the defect data.

8	Evaluate Defect Classes and Severity	4	**4**	Classification scheme with sub-classes; class, severity and priority for all ten; I7 vs I1 contrast explains severity ≠ priority.
9	Design and Appraise a Defect Repository	3	**3**	Field groups justified individually; analysis/improvement/prevention uses and e-mail vs repository comparison are convincing.
10	Evaluate Defect Examples and Prevention Strategies	4	**3.5**	Three classes, root causes and prevention strategies correct. Cost model rests on assumed ₹32,000 escape cost; overlapping savings noted but not quantified.
11	Recommend and Defend a Quality Improvement Plan	4	**3.5**	Prioritised, practical plan with 3.2% effort check. Expected effect depends on assumed 60%/35%/60% efficiencies that are not validated.
12A	Re-evaluate: Impact Analysis	2	**2**	Change table and affected assumptions clear; effect of two-week releases and shared records on severity well analysed.
12B	Re-evaluate: Evaluation of the Original Plan	2	**2**	Capacity check (184 vs 140 hours, 31% over) supports the conclusion; new external-system defect origins and repository gaps identified.
12C	Re-evaluate: Adaptation or Redesign	2	**2**	Clear MODIFY decision; revised two-week cycle, tester role, principles and external-defect classification (Table 26) shown.
12D	Re-evaluate: Evidence-Based Defence	2	**2**	Decision defended with defect, cost, capacity and patient-safety evidence; accepted risks and review trigger stated.
	TOTAL	**50**	**49**	**Percentage: 98%**

OVERALL REMARKS

Excellent, well-evidenced submission. Every task is answered with case data, the cost and defect-flow calculations are correct and internally consistent (₹26,10,500; 60 production escapes; 9.6% early detection), and the Day-4 redesign follows logically from a capacity calculation. Minor weaknesses: the savings in Tasks 10–11 depend on assumed review/test efficiencies and cost figures that are stated but not validated, and the solution repeats each task's bullet list, which adds length without adding marks.

P.Ranjithkumar		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.	✓	Tables 1–2 and incident mapping
2	A well-justified position on how process affects quality, backed by data from the case.	✓	TMM placement justified
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.	✓	Calculations shown; Figure 1 present
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.	✓	I3, I5, I6 and I8 covered
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.	✓	Six principles linked to evidence
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.	✓	Position defended with catch table
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.	✓	All ten incidents traced
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.	✓	Severity vs priority contrast given
9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.	✓	Each field group justified
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.	**P**	Savings rest on assumed costs
11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.	**P**	Efficiencies assumed, not validated
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence	✓	All four parts complete

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)	✓	Cost per phase and totals correct
Phase-containment / detection figures	✓	Table 7 flow; 40% detection before release

Origin assigned to every incident (I1–I10)	✓	I1–I10 with evidence and second origin
Class, severity and priority justified	✓	Table 17 with reasons
Defect repository fields with reasons	✓	Table 18 with reasons
Root cause → prevention strategy link	✓	Table 20
Cost–benefit of prevention strategies	**P**	Ratios 12.6 and 8.8 depend on assumptions
Interpretation of the defect pattern	✓	Pareto and investment conclusion
Figures and conclusions consistent	✓	Figures consistent across Tasks 1–12

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended	• Yes ☒ No

Software Testing (CSA3711)

ASSIGNMENT 1

CO1 Coverage: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository and defect examples.

Student Name	Register Number	Department / Section
P.Ranjithkumar	192411119	BE.CSE

Academic Year / Semester	Faculty Name	Date of Submission
2024	Dr.R kesavan	1.11.2026

Nature of Assignment: Individual **Duration:** 1 Week **Total Marks:** 50

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge: The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system. Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

Task 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity. Did the MediQueue team engineer quality into the product, or did they only hope for it?

Evidence Expected: A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

Ans:

Was MediQueue's Testing an Engineering Activity?

I judge the project against the four marks of engineering practice: work is **planned**, results are **measured**, activities follow **defined standards**, and the work is **repeatable** by someone else. For each mark I look for proof in the case. If the proof is missing, the mark is absent.

My verdict is that MediQueue was **not** tested as an engineering activity. The team built for eight months, checked its own code at the end, and released because a demonstration to management "showed no errors". A demonstration shows only the paths the presenter chooses to follow; it says nothing about the paths nobody tried. Quality was therefore hoped for, not designed in.

Table 1: MediQueue testing judged against the characteristics of an engineering activity

Characteristic	What engineering practice expects	Evidence from the case	Incidents it led to
Planning	Test objectives, levels, schedule, resources, risks and entry/exit criteria written down before testing starts	No test plan. Testing was done "just before release", after eight months of construction; no time, person or environment was set aside for it	I1, I6, I9, I10 (no concurrency, interface or performance test was ever planned)
Measurement	Coverage, defects per phase and cost are recorded and drive the release decision	The release criterion was a demo. Only 40 of 100 defects were found before release; patients found 60. No target existed that would show this to be unacceptable	All ten; explains why 60% of defects escaped

Characteristic	What engineering practice expects	Evidence from the case	Incidents it led to
Standards and defined process	Requirement and design reviews, a standard test-case format, traceability to requirements	Requirements and design had "no formal review"; coding had only a "developer self-check". The Sunday and GST rules were never documented	I2, I3, I7
Repeatability	Written test cases, test data and expected results that another person can rerun after every change	A demo cannot be rerun. The one documented test case mentioned (refund) was passed against a wrong expected value	I4, I5, I8
Independence and roles (supporting)	A person whose job is to find defects, with testing skills	No tester. Developers checked their own work, so the same wrong assumption was used to write and to "test" the code	I1, I5, I6
Tools and environment (supporting)	Test bed, test data, defect tracking, automation for repeated tests	Complaints held in e-mails and a spreadsheet; no test bed for SMS provider or load	I9, I10

How each missing feature led to incidents

- **No planning:** nothing forced the question "what happens on a slow network, with two users in the same second, or after the SMS key changes?". I1, I6, I9 and I10 are exactly the failures that appear only when such tests are planned.
- **No measurement:** with no exit criterion the team could not see its risk. Only 5 of the 52 requirement and design defects (2 + 3) were found early, and the testing phase caught 20 of the 75 defects then present (26.7%). A simple measure would have blocked the release.
- **No standards or reviews:** a missing working-day rule (I2) and a misread tax rule (I3) are found only when someone checks the requirement with the person who owns the rule. The wrong menu name in the manual (I7) is the same gap in document review.
- **No repeatability:** without written cases and expected values, a 41-character name (I4) or a patient aged exactly 60 (I5) is tested by luck. When a case was written (refund), its expected value was wrong and nobody rechecked it (I8).
- **No independent tester:** a developer who believes "two doctors never save together" will never write that test (I6).

What an engineered approach would have looked like

- A short test policy and a test plan per release: goals, levels (unit, integration, system, acceptance), schedule and risks (money, privacy, concurrency).
- Exit criteria, for example: all planned high-risk tests run, no open critical or high defect, at least 95% of planned tests passed, and hospital sign-off on the evidence.
- Reviews of requirements (with billing and OPD staff) and of the design (shared data, concurrency, interfaces) before coding.
- Test cases derived by equivalence partitioning and boundary values (name length 40/41; age 59/60/61), each with an expected result taken from the requirement and traced to it.
- System tests for concurrent booking and saving, performance on the hospital Wi-Fi, and the SMS interface, run on a proper test bed.
- A defect repository with simple measures (defects by phase, cost, escapes) reviewed after every release.

Conclusion: testing in MediQueue was an event, not an engineering process. Every missing feature maps to at least one incident, so the incidents were the predictable result of how the work was organised, not bad luck.

Task 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem. Is quality mainly a result of good people, or of a good process?

Evidence Expected: A justified position on the role of process in quality, supported by the case data.

Ans:

Good People or Good Process?

My position: MediQueue's problem is mainly a **process** problem. Better developers would help, but most of the defects arose before coding or outside the code, where programming skill cannot reach.

Table 2: Could a better developer alone have prevented each incident?

Incident	Prevented by a better developer alone?	Reason
I1, I6	Possibly	An experienced developer may think of concurrency, but only a design review and a concurrent-use test make it certain
I4, I5	Probably	Input length and age boundaries are coding skills, yet a boundary test would catch them even for an average developer
I10	Partly	Needs a performance target (a requirement) and a load test as well
I2, I3	No	The Sunday rule was never written and the GST rule was misread; skill cannot supply a rule the developer never received
I7	No	A documentation defect; needs a document review
I8	No	A defect in testing; needs test-case review and an independent tester
I9	No	The provider changed its key; needs interface monitoring and configuration control

Skill alone could have prevented at most 4 or 5 of the 10 incidents. The defect data agree: only 38 of 100 defects were introduced in coding, while 52 came from requirements and design. Even if better developers halved coding defects, 19 would disappear and 81 would remain. Hiring better developers therefore addresses under one fifth of the problem, whereas a defined process (reviews, planned tests, an independent tester, a repository) acts on every phase and also catches the slips of good developers.

Where a process-maturity view places the team

On the Testing Maturity Model (TMM) I place the team at **Level 1 (Initial)**: testing is mixed up with debugging, its purpose is only to show that the software works (the demo), and there is no test plan or trained tester. The immediate target should be **Level 2 (Phase Definition)**, whose goals fit the gaps exactly: separate testing from debugging, start test planning and apply basic techniques. **Level 3 (Integration)**, with a test group and a test life cycle integrated into development, would follow.

Task 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release. What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Evidence Expected: A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

Ans:

Testing as a Process Across the Life Cycle

I first describe the test process as it would apply to MediQueue, then place each incident in the phase where it should have been caught, and finally use the defect data to calculate the cost of late detection and the saving that reviews would bring.

1. Testing viewed as a process

Table 3: Test process activities applied to MediQueue

Activity	What it involves in MediQueue	Output
Planning	Set goals, levels, risks (concurrency, money, privacy), schedule and exit criteria	Test plan
Design	Derive cases from requirements: age 59/60/61, name length 40/41/42, Sunday booking, fee shown = fee charged, two bookings in the same second	Test cases with expected results
Set-up	Test bed with test doctors and patients, an SMS sandbox and a Wi-Fi condition like the hospital's	Test environment
Execution	Run planned tests at unit, integration and system level; rerun after every fix	Test log
Evaluation	Compare actual with expected results (the oracle); decide pass/fail; check exit criteria	Verdicts and defect reports
Reporting and closure	Defect reports, test summary and lessons learnt stored in the repository	Summary report and repository entries

2. Verification and validation in each phase

Table 4: Verification and validation activities for each development phase

Phase	Verification (building it right?)	Validation (building the right thing?)	Defects targeted
Requirements	Checklist review of requirements; testability check	Acceptance tests planned with hospital staff	I2, I3, I10
Design	Design inspection (concurrency, data handling, interfaces)	System test cases designed	I1, I6, I9
Coding	Code review, coding standard, boundary-value unit tests	–	I4, I5
Integration and system test	Interface tests, regression tests, test-case review	System tests (performance, concurrency) and user acceptance test	I1, I3, I6, I9, I10
Documentation	Manual checked against the working application	A real user follows the manual	I7
Operation	Interface monitoring; regression after every change	Patient feedback	I9

3. Phase in which each incident ought to have been detected

Table 5: Intended detection phase for each incident

Incident	Defect	Phase it should have been caught	Activity that would reveal it
I1	Same token issued to two patients	Design	Design inspection of token allocation; concurrent-booking system test
I2	Sunday rule missing	Requirements	Requirement review with OPD staff
I3	₹500 shown, ₹550 charged	Requirements	Review with billing; system test comparing displayed and charged fee
I4	Crash for names over 40 characters	Coding	Boundary-value unit test (40, 41)
I5	No senior priority at age 60	Coding	Boundary test (59, 60, 61); code review
I6	Prescription shows wrong patient	Design	Design inspection of shared data; concurrent-save test
I7	Manual names a non-existent menu	Documentation	Review of manual against the application
I8	Wrong expected value in refund test	Testing	Second-person review of the test case
I9	SMS stops after key change	Design / integration	Interface test, configuration control, production monitoring
I10	Schedule screen takes 25 s	Requirements / system test	Performance requirement; load test on hospital Wi-Fi

Under this plan five incidents (I1, I2, I3, I6, I10) belong to the requirements and design phases, two (I4, I5) to coding, and three to testing, documentation and operation. In the real project, patients found all ten.

4. The cost of late detection to the hospital

Table 6: Cost of the last release's 100 defects, using the unit costs in the case

Phase of detection	Defects detected	Cost per defect (₹)	Total cost (₹)	Share of total cost
Requirements	2	500	1,000	0.04%
Design	3	1,500	4,500	0.2%
Coding	15	3,000	45,000	1.7%
Testing	20	8,000	1,60,000	6.1%
Production (after release)	60	40,000	24,00,000	91.9%
Total	**100**	**avg 26,105**	**26,10,500**	**100%**

Calculation: production 60 × 40,000 = ₹24,00,000; testing 20 × 8,000 = ₹1,60,000; coding 15 × 3,000 = ₹45,000; design 3 × 1,500 = ₹4,500; requirements 2 × 500 = ₹1,000. Sum = **₹26,10,500**, an average of ₹26,105 per defect.

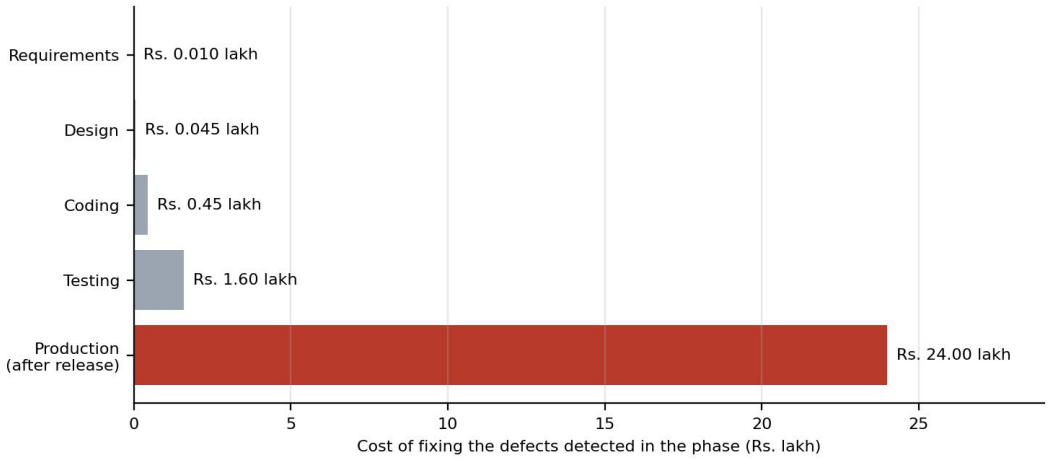


Figure 1: Defect cost by the phase in which the defects were detected

Interpretation: patients found 60% of the defects, and those defects produced 91.9% of the cost. One production defect costs 80 times a requirements defect (40,000 ÷ 500) and 5 times a testing defect (40,000 ÷ 8,000). If the 60 production defects had been found in testing at ₹8,000, they would have cost ₹4,80,000 instead of ₹24,00,000, a gap of ₹19,20,000 (an upper limit, since testing cannot catch everything). If every one of the 100 defects had been fixed in the phase that introduced it, the cost of the 95 defects with a stated unit cost would be 30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 = ₹2,02,000. Late detection therefore added about ₹24 lakh (26,10,500 − 2,02,000), roughly twelve times the ideal cost.

Table 7: Defects caught by each phase out of those present in that phase

Phase	Introduced	Present in phase	Detected	Detection efficiency	Passed on
Requirements	30	30	2	6.7%	28
Design	22	50	3	6.0%	47
Coding	38	85	15	17.6%	70
Testing	5	75	20	26.7%	55
Documentation	5	60	0	0.0%	60
Production	–	60	60	100%	0

Defects present = defects passed on from the previous phase + new defects. The 60 defects left after documentation match the 60 found in production. Requirements and design introduced 52 defects but detected only 5 (9.6%). Only 40 of the 100 defects were found before release, so the "no errors" decision was made while 60 defects were still hidden in the product.

5. Probable saving if requirements and design reviews had been done

Using the flow in Table 7, I assume a requirements review and a design review each remove a fraction r of the defects present in their phase (instead of today's 6.7% and 6.0%), while coding and testing keep their present rates (17.6% and 26.7%). Defects removed early are charged at that phase's cost; the rest move on. I tried r = 40%, 60% and 80%, and assumed both review programmes cost ₹1,00,000 (about 125 person-hours at ₹800 per hour, including preparation).

Table 8: Estimated savings from requirements and design reviews

Review efficiency	Found by reviews	Defects reaching patients	Cost after reviews (₹)	Gross saving (₹)	Review cost (₹)	Net saving (₹)	Net saving %
40%	28.0	46.1	20,26,808	5,83,692	1,00,000	4,83,692	18.5%

Review efficiency	Found by reviews	Defects reaching patients	Cost after reviews (₹)	Gross saving (₹)	Review cost (₹)	Net saving (₹)	Net saving %
60%	38.4	39.8	17,61,399	8,49,101	1,00,000	7,49,101	28.7%
80%	46.4	35.0	15,55,854	10,54,646	1,00,000	9,54,646	36.6%

With 60%-efficient reviews, defects reaching patients fall from 60 to about 40, cost falls from ₹26,10,500 to about ₹17,61,000, and the net saving is about **₹7.5 lakh (28.7%)** after paying for the reviews. Even at 40% efficiency the net saving is ₹4.8 lakh, because ₹1 lakh of review effort avoids many ₹40,000 production fixes. In the same model, a testing phase that catches 60% of defects present (instead of 26.7%) would cut production defects from 60 to 35 and save about ₹8 lakh, so reviews and better testing complement each other.

6. Limitations and assumptions

- **Defect flow:** the case does not say which phase each detected defect came from, so I assumed that defects missed in one phase pass to the next (Table 7). The true flow may differ.
- **Review efficiency:** the 40–80% range is an assumption because the team has no review data.
- **Unit costs:** the case figures cover rework only. They leave out patient harm, loss of trust, legal exposure and staff time, so the real cost of production defects is higher.
- **Documentation defects:** no cost is given; I treated the five remaining ones as production defects.
- **Single release:** the data come from one release of one team; undiscovered defects are not counted.
- **Review cost:** ₹1,00,000 is my own estimate; a higher cost would shrink the benefit but not remove it.

Task 4 – Apply and Critique Basic Definitions **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports. Does careless terminology actually harm quality, or is it only a matter of words?

Evidence Expected: Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

Ans:

Applying and Critiquing the Basic Definitions

Terminology is not just words. When "bug", "error" and "failure" mean the same thing, nobody can tell whether the problem sits with a person, in the product, or in what the user sees, so the cause cannot be traced or prevented. I apply each term to MediQueue and then to three incidents.

Table 9: Basic definitions in the MediQueue context

Term	Meaning	MediQueue example
Error	A mistake made by a person	A developer assumes only one doctor saves a prescription at a time
Fault (defect, bug)	A flaw in the software or a document, caused by an error	The PDF generator keeps patient data in a variable shared by all doctors

Term	Meaning	MediQueue example
Failure	Incorrect behaviour the user observes when faulty code runs	A prescription PDF shows the previous patient's details
Test case	Inputs, conditions and expected result for one check	Book a Sunday appointment → system must reject it and show "OPD closed"
Test oracle	The source that decides the expected result (requirement, rule, calculation, expert)	The approved fee table (₹500 + tax) decides whether ₹550 is correct
Test bed	The environment in which tests run: hardware, network, data, tools, stubs	A copy of the app with test doctors and patients, an SMS sandbox and hospital Wi-Fi conditions

Table 10: Error, fault and failure for incidents I3, I5 and I6

Incident	Error (human mistake)	Fault (defect in the product)	Failure (what the user saw)
I3	The analyst or developer misread the GST rule and implemented it differently in two places	The booking page shows the base fee while the payment module adds tax: two separate fee calculations	Page shows ₹500; patient pays ₹550
I5	The developer read "senior citizen" as "older than 60", or compared only the year instead of the full date	A condition such as age > 60, or age computed on the booking date instead of the appointment date	A patient turning 60 on the appointment day gets no priority
I6	The developer assumed two doctors would never save at the same moment	Patient details held in shared data that the second doctor overwrites before the PDF is made	The PDF shows the previous patient's details

How terminology confusion allowed I8

In I8 the tester recorded a refund test as "Pass" after comparing the output with a wrong expected value. The tester's **error** put a **fault** into the test case (a wrong oracle), and the fault produced a false verdict. A team that calls everything a "bug" cannot separate a fault in the product from a fault in the test, and never asks where the expected value came from. No oracle had been defined, so the value was written from memory instead of from the requirement, and the "Pass" was believed. The refund problem therefore stayed hidden behind a wrong test.

Does loose terminology damage quality?

Yes, in four ways: (1) **root-cause analysis breaks down**, since a failure comes from a fault and a fault from an error, and each needs a different cure (code fix, review checklist, training); (2) **measurement is distorted**, because the 100-defect count mixes patient-reported failures with code faults, so one fault may be counted many times; (3) **communication with management suffers**, as "bug" says nothing about severity; (4) **fix verification is weak**, because nobody records which test case and oracle prove the fault is gone.

A shared glossary

The team needs a one-page glossary agreed by developers, testers and hospital staff and kept next to the defect repository.

Table 11: Minimum contents of the MediQueue glossary

Group	Terms to define	Why it is needed
Defect terms	Error, fault/defect, failure, incident	Separates cause, flaw and symptom in every report
Test terms	Test case, expected result, test oracle, test bed, test log, regression test	Makes tests repeatable and prevents wrong expected values (I8)
Rating terms	Severity and priority levels with MediQueue examples	Ends arguments about what to fix first

Group	Terms to define	Why it is needed
Defect states	New, assigned, fixed, verified, closed, reopened	Tracks each defect to closure
Product terms	Build, release, hotfix, appointment day, senior citizen, fee, token	Removes ambiguity such as "senior at 60" (I5)

Task 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct". Which testing principles prove this claim wrong?

Evidence Expected: A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

Ans:

Which Principles Show the Claim Is Wrong?

The claim makes two mistakes: it treats "no failures in the tests we ran" as proof of correctness, and it assumes every possible test was run. I apply six principles and tie each one to the case.

Table 12: Testing principles applied to MediQueue

Principle	Violated / confirmed	Evidence from the case
Testing must be planned	Violated	No test plan; testing happened just before release, so nothing was set aside for load, concurrency or the SMS interface (I10, I1, I6, I9)
Testing should span the whole life cycle (start early)	Violated	Requirements and design were never reviewed; 52 of 100 defects began there and only 5 were caught early. Late detection cost ₹26.1 lakh, 91.9% of it in production
Test valid and invalid / extreme inputs	Violated	A 41-character name (I4) and a Sunday booking when the OPD is closed (I2) are extreme or invalid inputs the demo never tried
Every test case needs an expected result that is checked carefully	Violated	The refund test was passed against a wrong expected value (I8)
A good test is one likely to find a new defect	Violated	The "test" was a management demo meant to show success; it found nothing because it was never designed to
Testing should be done by an independent, qualified person	Violated	Developers tested their own code, so the same assumption (single user, "age > 60") sat in both code and tests (I5, I6)
Defects cluster	Confirmed	I1 and I6 share one cause (shared data under simultaneous use); after the first was found, the second should have been searched for

A further principle is that testing can show the presence of defects but never their absence. The developers read an absence of failures in a few demo cases as proof of correctness, which this principle rules out. Exhaustive testing is also impossible, so tests must be chosen by risk and technique; no such choice was made.

The principle with the greatest impact

In my judgement, **"testing should span the whole life cycle and start early"** would have had the greatest effect. Half the incidents (I1, I2, I3, I6, I10) come from requirements and design, those phases caught only 9.6% of their own defects, and the cost per defect climbs from ₹500 in requirements to ₹40,000 in production. Planning and independence help, but both work far

better when they begin at the requirements stage. Test cases written from the requirements would have exposed the missing Sunday rule and the fee mismatch before any code existed.

Task 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester. Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Evidence Expected: A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

Ans:

Would an Independent Tester Have Made a Difference?

My position: an independent tester would have changed the outcome greatly, but not removed every defect. Working across the life cycle, a tester would very probably have caught seven of the ten incidents. The other three (I1, I6, I9) also need design review, specialised tests and production monitoring.

The tester's role across the life cycle

- **Requirements:** review for missing, unclear or untestable statements; ask "what about Sunday? is the fee inclusive of tax? how fast must the schedule load?".
- **Design and planning:** write the test plan, risks and exit criteria; design test cases and the test bed.
- **Coding and integration:** support unit testing, run integration tests, maintain the regression suite.
- **System and acceptance:** run functional, load, concurrency and security tests; support user acceptance.
- **Throughout:** report defects with reproduction steps, verify fixes, maintain the repository and give management evidence for the release decision.

Developers testing their own code

Advantages: the developer knows the code, finds simple faults quickly and fixes them cheaply; developer unit testing is valuable and should continue. **Limitations:** the developer tests what he or she believes the code does, so a wrong assumption is carried into the test; there is a natural wish to show the work is good; deadlines push testing to the end; and nobody checks the developer's reading of the requirement. I5 (the age-60 boundary) and I6 (the single-user assumption) both come from the developer's own assumptions.

Table 13: Incidents a tester would most probably have caught

Incident	Detected by a tester?	Stage and method
I2	Yes	Requirement review / test design: ask for the OPD working days
I3	Yes	Requirement review; system test comparing displayed fee with amount charged
I4	Yes	Unit or system test with boundary-length names (40 and 41 characters)
I5	Yes	Test design with boundary ages 59, 60, 61 on the appointment day
I7	Yes	Document review: follow the manual step by step on the real application
I8	Yes	Second-person review of the test case; oracle traced to the requirement

Incident	Detected by a tester?	Stage and method
I10	Yes	System load test on hospital Wi-Fi once a performance target is agreed
I1	Maybe	Only if a concurrency test is designed (needs tools and skill); design inspection is the stronger safeguard
I6	Maybe	As for I1; needs a concurrent-save test and a design review of shared data
I9	Partly	Interface test with an SMS sandbox; the later key change needs monitoring and configuration control

The result is 7 "yes", 2 "maybe" and 1 "partly". In terms of the defect data, a tester who lifted testing-phase detection from 26.7% to 60% (Task 3) would cut production defects from 60 to about 35. That is a large gain, but 35 defects would still reach patients, so a tester alone is not enough, which is why my plan also includes reviews.

Qualities of a tester for a healthcare application

- **Domain knowledge:** outpatient workflow, fees and tax, senior-citizen rules, prescriptions.
- **Eye for detail and curiosity:** notices a ₹50 difference, a wrong menu name or a wrong expected value.
- **Sensitivity to patient safety and privacy:** knows that showing one patient's data to another (I6) is a serious breach.
- **Technical skill:** test design techniques, concurrency and load testing, defect tracking, basic automation.
- **Communication and ethics:** reports defects clearly and without blame, protects health data, and can say "not ready for release".

Task 7 – Trace the Origins of Defects **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated. Where in the life cycle are MediQueue's defects really coming from?

Evidence Expected: A justified origin for each incident and an interpretation of the overall pattern of defect origins.

Ans:

Where Do MediQueue's Defects Come From?

For each incident I assign one main origin, quote the evidence in the case, and name a possible second origin with my reason for preferring the main one.

Table 14: Origin of every incident

Incident	Main origin	Supporting evidence from the case	Possible second origin and why I chose the main one
I1	Design	Duplicate tokens appear only for same-second bookings; token allocation was not designed as one indivisible step	Coding. Chosen Design because the flaw is in how allocation is structured, not a typing or logic slip
I2	Requirements	The rule "was never written in the requirements"	None
I3	Requirements	The GST rule was "misunderstood": the business rule was unclear or wrong at requirements stage	Coding. Chosen Requirements because the misunderstanding concerns a business rule, which is requirement knowledge

Incident	Main origin	Supporting evidence from the case	Possible second origin and why I chose the main one
I4	Coding	Crash on names over 40 characters: length handling is missing	Requirements (no maximum length stated). Chosen Coding because a program must not crash on any input whatever the specification says
I5	Coding	The “age 60” rule is simple; only its boundary was coded wrongly	Requirements (“turns 60” date not stated). Chosen Coding because the rule itself is common knowledge
I6	Design	Two doctors overwrite shared data; sharing state is a design decision	Coding. Chosen Design because data sharing is an architectural choice
I7	Documentation	The manual refers to a “My Visits” menu that does not exist	None
I8	Testing	The tester checked the output against a wrong expected value	None
I9	Environment / design	The SMS provider changed its key; the application neither handled nor monitored the external change	Design (no error handling or alert). Environment is main because the trigger is outside the team, but the design weakness let it cause harm
I10	Requirements	No performance target is stated anywhere in the case	Design or coding (slow loading). Chosen Requirements because without a target nobody could call 25 seconds a defect

Table 15: Overall pattern of origins

Origin	Incidents	Count	Share of 10 incidents
Requirements	I2, I3, I10	3	30%
Design	I1, I6	2	20%
Coding	I4, I5	2	20%
Testing	I8	1	10%
Documentation	I7	1	10%
Environment	I9	1	10%

Interpretation: 5 of 10 incidents (50%) arise in requirements and design, which matches the release data (52 of 100 defects introduced there). Only 2 incidents (20%) come from coding, yet the team put all its (self-check) effort into coding. Three of the ten are not code defects at all (testing, documentation, environment). I1 and I6 share one cause, shared data used simultaneously, so the same design weakness may exist elsewhere.

What this means for the hospital: invest first in requirement reviews, design inspections and an independent test activity, not in more coding skill. These are also the cheapest places to remove defects (₹500 and ₹1,500 per defect against ₹40,000 in production).

Task 8 – Evaluate Defect Classes and Severity **4 Marks**

CO1 | Bloom’s Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects. Which classes of defects are most harmful to patients, and do the team’s priorities reflect that?

Evidence Expected: A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

Ans:

Defect Classes, Severity and Priority

I use the four standard classes (requirements/specification, design, coding, testing) with their sub-classes. Two incidents do not fit them (the user manual and the external SMS provider), so I add a documentation class and treat the external provider as an external-interface sub-class of design.

Table 16: Defect classification scheme applied to MediQueue

Class	Sub-classes	Examples
Requirements / specification	Functional description, feature, business rule, interface description, non-functional (performance) requirement	I2, I3, I10
Design	Algorithm/processing, control logic and sequence, data and shared state, module interface, external interface	I1, I6, I9
Coding	Algorithm/processing, control logic (boundary), data/input handling, initialisation, typographical, code documentation	I4, I5
Testing	Test-case design, test oracle (expected result), test procedure, test harness	I8
Documentation (added)	User manual, help text, release notes	I7

Scales used. Severity = impact on patients and hospital: Critical – risk to patient safety or privacy, or money lost by many patients; High – a key function wrong or unusable; Medium – wrong but a workaround exists; Low – cosmetic or informational. Priority = the order of fixing, based on severity combined with how often it occurs, workaround, cost of the fix and risk of delay.

Table 17: Class, severity and priority of every incident

Incident	Class / sub-class	Severity	Priority	Reason
I1	Design / control logic (concurrency)	High	Medium	Shared tokens disturb the queue, but it needs the same second and doctor, and reception can issue a new token, so it follows the urgent items
I2	Requirements / business rule (missing)	Medium	High	Patients travel to a closed OPD but safety is unaffected; the fix is a cheap rule and many people are affected, so it is done early
I3	Requirements / business rule (wrong)	High	Critical (first)	Patients are overcharged on every booking; money, trust and possible tax or legal exposure
I4	Coding / input handling	High	Medium	The app crashes, but only for names over 40 characters; a shorter name is a workaround and the fix is easy
I5	Coding / control logic (boundary)	Medium	Medium	A small group loses priority on its birthday; a fairness and care issue
I6	Design / data and shared state	Critical	Critical (first)	One patient's prescription shows another's details: a patient-safety and privacy breach
I7	Documentation / user manual	Low	High	No data affected, but every patient who follows the manual is misled; the fix takes minutes
I8	Testing / test oracle	High	High	The false "Pass" hides a possible refund defect and casts doubt on all test results; money at stake
I9	Design / external interface	Medium	Medium	Missed reminders cause missed visits but no harm; needs alerting and key handling
I10	Requirements / non-functional (performance)	Medium	Medium	Slow but accurate; needs a target and profiling, so it is planned for the next cycle

Why severity and priority can differ

Severity is the damage if the defect occurs; priority is how soon it should be fixed, which also depends on frequency, workaround, cost of the fix and competing work. **I7** is Low in severity but High in priority: it damages no data, yet every patient who reads the manual is misled and the fix takes minutes. **I1** is High in severity but only Medium in priority: it is serious, but it needs two bookings in the same second and reception can issue a new token, and I3 and I6 are waiting. A priority list based on severity alone would therefore be wrong.

Most harmful class and where to start

The most harmful defects for patients are the shared-data design defect (I6, Critical) and the money-related requirement defect (I3). The team's own priorities do not reflect this: it looked only at visible errors during a demo and had no severity scale, so a crash and a privacy leak would have been treated alike.

I recommend addressing the **requirements/specification class first**, for three reasons: (1) it holds 30 of the last release's 100 defects and 3 of the 10 incidents; (2) only 2 of those 30 were caught before release (6.7% detection); (3) it is the cheapest class to fix early (₹500 against ₹40,000 in production). Design inspection of shared data and concurrency should follow immediately, since that class holds the Critical defect I6.

Task 9 – Design and Appraise a Defect Repository — 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet. What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Evidence Expected: A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

Ans:

A Defect Repository for MediQueue

A defect repository is the project's memory. For each defect it must let the team answer: what went wrong, where it started, why it was not caught earlier, how serious it was, how it was fixed and how to stop it recurring.

Table 18: MediQueue defect repository fields and the reason for each

Field group	Fields	Reason it is needed
Identification	Defect ID, title, date reported, reporter, source (patient complaint, tester, review, monitoring)	Unique reference; shows who finds defects and how, exposing weak detection
Description	Description, reproduction steps, expected result, actual result, attachments	Lets a developer reproduce and fix without more questions; separates the failure from expected behaviour
Location	Module/feature, release and build, environment (production/test), browser/device	Shows where defects cluster and which release holds them
Classification	Class and sub-class (requirements, design, coding, testing, documentation, environment)	Basis for analysing defect types and building checklists
Origin and detection	Phase introduced, phase detected, detection method (review, unit test, system test, production)	Gives the phase-containment figures used in Task 3 and the escape rate

Field group	Fields	Reason it is needed
Rating	Severity, priority, patient-safety/privacy flag, number of patients affected	Orders the work; highlights serious risks
Handling	Status (new, assigned, fixed, verified, closed, reopened), owner, dates	Tracks each defect to closure; allows ageing reports
Cause and fix	Root cause (the error), fix description, effort (hours), cost	Supports cost-per-phase figures and learning from causes
Traceability	Requirement ID, test case ID, related or duplicate defects	Shows which requirement or test was missing; links recurring defects
Prevention	Proposed preventive action, owner, status	Turns every defect into a process improvement

How the repository supports analysis, improvement and prevention

- **Defect analysis:** Pareto charts by class, module and phase show where to focus; origin and detection counts supply the Task 3 figures for every release.
- **Process improvement:** a rising share of requirement defects found late shows reviews are ineffective; many reopened defects point to weak fixing or verification; the data set targets (for example, production escapes below 20%).
- **Prevention:** every cause becomes an action (checklist item, coding rule, regression test). I1 and I6 would produce a design-checklist entry "shared data under simultaneous use" and a concurrency test for each release.
- **Test design:** past defects supply regression cases and new test ideas for similar modules.
- **Management:** trend figures replace "the demo went well" as the basis for release decisions.

Shortcomings of the present e-mail and spreadsheet method

Table 19: E-mail and spreadsheet compared with a repository

Aspect	E-mails and spreadsheet	Defect repository
Structure	Free text; people use different words and columns	Fixed fields and lists; common terms
Duplicates	The same problem is reported again and again unrecognised	Duplicates linked to one defect
Workflow	No status or owner; defects get lost in mailboxes	Status, owner, due dates, reminders
History and audit	No change history; spreadsheet versions conflict	Every change logged with date and user
Analysis	Counting by class, phase or cost needs manual work	Built-in reports and filters
Traceability	No link to requirement, test case or build	Linked to all three
Privacy	Patient names and health details circulate in e-mails and shared sheets	Access control and masking of patient data

In my judgement the repository pays for itself: a free or low-cost tracker is enough, filling the fields takes a few minutes per defect, and in return the hospital gets the data to decide where to invest, which it could not get before.

Task 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes. For each selected defect, what single change to the process would have prevented it?

Evidence Expected: A root-cause analysis of three defects with justified, cost-aware prevention strategies.

Ans:

Root Causes and Prevention Strategies for Three Defects

I chose one incident from each of three classes: **I3** (requirements), **I6** (design) and **I5** (coding). For each I give the root cause, one process change that would have prevented it, and the cost against the benefit.

Table 20: Root cause and prevention strategy for the chosen defects

Incident (class)	Root cause	Prevention strategy (one process change)
I3 – wrong fee charged (requirements)	The fee and tax rule was never documented or agreed with billing; it was implemented twice (display and payment) from different readings	Requirement review with billing staff and a written fee-rule table; fee calculated in one place; tests use the table as oracle
I6 – wrong patient on prescription (design)	Data was shared across users and nobody examined the design for simultaneous use; no design review	Design inspection with the checklist item "shared data and simultaneous users", plus a concurrency test for saving prescriptions
I5 – no senior priority at 60 (coding)	The age rule was coded without a boundary check; nobody else read the code or tested 59/60/61	Coding standard for date and age calculations, code review, and unit tests at boundary values 59, 60, 61 for every rule

Cost assumptions. One person-hour costs ₹800. A defect that escapes its own phase costs on average about ₹32,000 to fix later, being the mean of the testing and production figures: (1,60,000 + 24,00,000) ÷ 80 defects. Extra defects caught by each strategy come from the Task 3 model (reviews at 60% efficiency) and, for coding, from raising coding-phase detection from 17.6% to 35% of the 85 defects present.

Table 21: Cost and benefit of each strategy for one release

Strategy	Effort (person-hours)	Cost (₹)	Extra defects caught early	Saving per defect (₹)	Total saving (₹)	Benefit ÷ cost
Requirement review (all requirements)	50	40,000	16 (18 instead of 2)	31,500 (32,000 – 500)	5,04,000	12.6
Design inspection	75	60,000	17.4 (20.4 instead of 3)	30,500 (32,000 – 1,500)	5,30,700	8.8
Coding standard + code review + boundary unit tests	80	64,000	14.75	29,000 (32,000 – 3,000)	4,27,750	6.7

All three strategies are worthwhile, since each returns over six times its cost. The savings overlap (a defect can be caught only once), so they cannot simply be added, but the ranking is clear.

Best value: the requirement review. It has the highest benefit-to-cost ratio (about 12.6), needs only a few hours of discussion with billing and OPD staff, requires no tools, and targets the largest group of defects (30 of 100). It would have prevented I3, I2 and I10, and it underpins the other strategies because test oracles come from reviewed requirements. The design

inspection has a lower ratio (about 8.8) but I would still keep it, since it addresses the Critical defect I6, whose real cost (a privacy breach) is far above the ₹40,000 used in the model.

Task 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Evidence Expected: A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

Ans:

Quality Improvement Plan for MediQueue

The plan follows from the evidence: half the incidents (and 52% of defects) begin in requirements and design; only 9.6% of those are caught early; production defects carry 91.9% of the cost; and there is no independent tester, exit criterion or repository. Changes are ordered by value for money and by what five developers can realistically do.

Table 22: Quality improvement plan in priority order

Priority	Change	Role of tester / reviews / repository	Addresses	Effort
1	Release gate: written exit criteria (no open critical or high defect, planned high-risk tests done, at least 95% passed) signed off by the hospital	Tester gathers the evidence; management decides	Demo-based approval; all incidents	A few hours per release
2	Review every rule (fees, working days, age rules, performance target) with billing and OPD staff; keep a business-rule table	Tester and developers review; the table is the test oracle	I2, I3, I5, I10	About 50 person-hours
3	Appoint an independent tester (full-time, or a trained hospital IT staff member)	Writes test plan, designs tests (equivalence classes, boundaries), runs system tests, reviews test cases (I8)	I4, I5, I7, I8, I10	About 8 months of one person
4	Design inspection of high-risk areas (shared data, concurrency, external interfaces); add concurrency and load tests	Tester and two developers; checklist kept in repository	I1, I6, I9	About 75 person-hours
5	Defect repository with the Task 9 fields; monthly review by class, phase and cost	Whole team; tester owns the reports	Learning from every incident	1 day set-up, 5 minutes per defect
6	Coding standard, peer review and boundary-value unit tests	Developers	I4, I5	About 80 person-hours
7	Walk through the manual on the live application; interface monitoring and alerts for the SMS provider	Tester; operations	I7, I9	A few hours

Expected impact on number and cost of defects

I applied the Task 3 flow model with these assumptions: reviews of requirements and design remove 60% of defects present; coding-phase detection rises from 17.6% to 35% (code review and unit tests); testing-phase detection rises from 26.7% to 60% (planned tests and an independent tester).

Table 23: Expected outcome for one release of the same size as the last

Measure	Last release	With the plan
Defects detected before release	40	79.6
Defects reaching patients	60	20.4
Cost of fixing the defects (₹)	26,10,500	10,95,412
Cost of the plan: tester ₹4,00,000; reviews ₹1,00,000; code review and unit tests ₹64,000; training and tools ₹50,000 (₹)	–	6,14,000
Total cost (₹)	26,10,500	17,09,412
Net saving (₹)	–	**9,01,088 (34.5%)**

The plan cuts defects reaching patients by about two thirds (60 to about 20) and saves about ₹9 lakh per release after paying for itself, without counting unquantified benefits such as fewer patient-safety and privacy incidents.

Trade-offs, risks and limitations

- **Slower first release:** reviews and testing take calendar time; the hospital must accept a gate that may delay a release.
- **Assumed efficiencies:** the 60%, 35% and 60% figures are estimates; poor reviews would shrink the saving. I would record real figures in the repository and update the model.
- **Review as ritual:** reviews can become box-ticking; checklists and recorded defects keep them honest.
- **Tester–developer friction:** the tester must be seen as part of the team and defects reported without blame.
- **Costs not counted:** patient harm and loss of trust are outside the figures, so the real benefit is larger.
- **Single-release data:** the estimate rests on one release of 100 defects.

Is it workable for five developers?

Yes, if kept lightweight. Extra developer effort is about 125 person-hours for the two review programmes plus 80 for code review and unit tests, about 205 hours. Five developers over eight months provide 5 × 8 months × 20 days × 8 hours = 6,400 hours, so the plan uses about **3.2%** of developer time, plus one tester. Reviews are short (90 minutes), checklist-driven and aimed at the highest-risk parts, and the repository tool can be free. The one large cost is the tester, who is also the largest source of savings.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement: The hospital now wants to release a new version of MediQueue every two weeks instead of every six months. At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency. Patient health records will now be shared between hospitals. Re-evaluate your original solution in response to these changed conditions. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis

CO1 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A clear analysis of how the changed conditions affect the original quality improvement plan.

Ans:

Impact of the New Conditions

Table 24: Original versus changed situation

Aspect	Original situation	Changed situation
Release rhythm	One release every six to eight months	A release every two weeks (26 a year, 13 times as often)
Interfaces	The team's own application plus an SMS provider	Also the national digital health ID system run by an external agency
Data	Appointment and prescription data of one hospital	Health records shared across hospitals
Control	The team controls its own code and releases	The agency controls when and how the ID system changes

Assumptions in my plan that are affected

- **One large review per release:** the plan assumes whole-document reviews (about 125 person-hours). In a two-week cycle this cannot be repeated each release; reviews must be small and tied to each change.
- **Manual testing by one tester:** a full manual regression run (say 400 cases × 6 minutes = 40 person-hours) would eat half the 80 hours one tester has in a two-week cycle.
- **A test bed we control:** with the external ID system, the test environment is the agency's sandbox, which may not behave like the live system.
- **A repository for one system:** it assumes every defect is in our code; some will now lie in the agency's system or in the contract between us.
- **₹40,000 per production defect:** this rises when a defect can pass wrong data to other hospitals.

Why frequent releases change the role of testing and the tester

With a release every two weeks there is no "testing phase at the end". Testing must run alongside development; a tester who joins only at the end becomes a permanent bottleneck. The tester joins from story refinement onward, spends less time on repeated checks (which are automated and run on every change) and more on risk analysis, exploratory testing, automation

and the release decision. Every release must also be reversible and monitored, because a faulty release is live for 6,000 patients a day.

Why shared health records raise the severity of some defect classes

Once records are shared, an I6-type defect (another patient's data in a document) stops being a local mistake and becomes a confidentiality breach across hospitals, with legal duties and possible harm to a patient treated on wrong information. Design and data defects (shared state, identity matching) and interface defects therefore become Critical, as would an access-control defect that was Medium before. Requirement defects about consent and identity also become Critical, whereas defects like I7 (manual) remain Low.

B. Evaluation of the Original Plan

CO1 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A technically supported evaluation of the suitability of the original plan under the new conditions.

Ans:

Is the Original Plan Still Suitable?

Table 25: Fit of the planned activities within a two-week cycle

Planned activity	Fits in two weeks?	Reason / change required
Review of all rules (about 50 person-hours)	No, not as it stands	Review only the rules in that cycle's stories (30–45 minutes per story); keep the rule table current
Design inspection (about 75 person-hours)	Partly	Do it only for stories involving shared data, identity or external interfaces; keep the checklist
Independent tester doing manual testing	No	About 114 + 40 (regression) + 30 (ID system and privacy checks) = 184 person-hours needed per cycle against about 140 available (tester 80 + 15% of the developers' 400 hours = 60): 31% over capacity
Release gate	Yes	Keep; automate the checks that feed it
Code review and unit testing	Yes	Keep as part of the definition of done
Defect repository	Partly	Needs new fields (below)

The capacity figures are my own estimates: the original plan's effort (about 1,485 person-hours per six-month release, being 1,280 for the tester, 125 for reviews and 80 for code review) divided by 13 for the smaller change in each release gives about 114, to which I add one full regression run and the new checks.

New defect origins brought by the external health ID system

- **Contract and version defects:** the agency changes a field, version or rule without notice (the same pattern as I9).
- **Data-mapping and identity defects:** patients matched wrongly, inconsistent name and date formats, duplicate IDs.
- **Availability and speed:** the external service is slow or down; our application freezes or loses the booking.
- **Authentication and consent defects:** expired tokens, missing patient consent, wrong access rights.
- **Environment differences:** the agency's test environment behaves differently from the live service.

- **Privacy and security defects:** records exposed in transit or in logs.

Is the defect repository still adequate?

No. It was built for defects in the team's own system. It needs extra fields: whether the defect is internal, in the agency's system or in the interface; the interface version or contract reference; the agency's ticket number and reply date; data sensitivity and number of patients or hospitals affected; whether it must be reported to an authority; the release and pipeline stage where it was found; and time to detect and to restore service. Without them the team cannot tell its own defects from the agency's or see how quickly problems are found and fixed.

C. Adaptation or Redesign

CO1 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: An appropriate response to the changed requirement with clear quality-engineering reasoning.

Ans:

Decision: Modify the Quality Process

My decision is to **MODIFY** the plan. I do not retain it unchanged, because Table 25 shows its reviews and manual testing cannot fit a two-week cycle (about 31% over capacity) and it has no way to handle an external system. I do not redesign it from scratch, because the evidence behind it still holds: over half the defects begin in requirements and design, early detection is far cheaper (₹500 or ₹1,500 against ₹40,000), and an independent tester, a release gate and a repository are still needed. What changes is how often and how small each activity is, and what gets automated.

The revised process

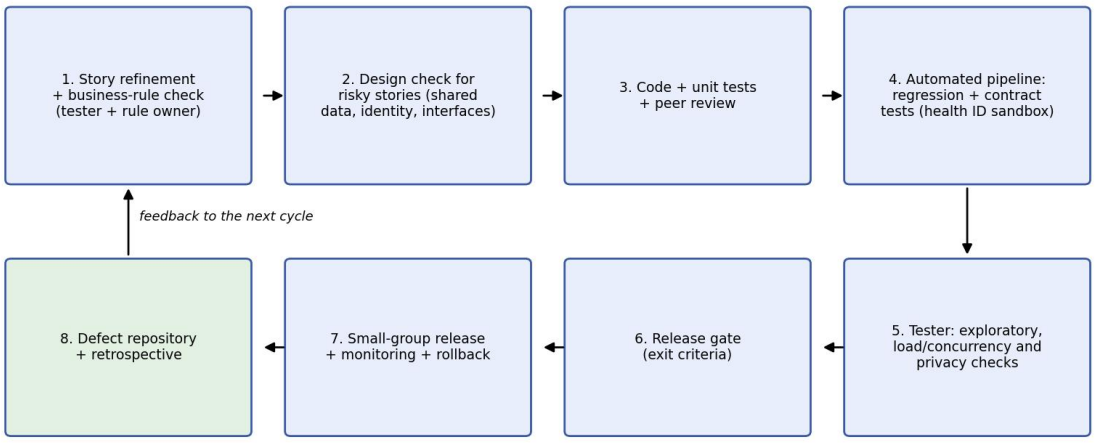


Figure 2: Revised two-week quality cycle

- **Story refinement:** every story gets acceptance criteria and a business-rule check by the tester and the hospital owner of that rule (30–45 minutes).
- **Design check:** only for high-risk stories (shared data, identity, external interface), using the checklist.
- **Development:** peer review and boundary-value unit tests are part of the definition of done.

- **Automated pipeline:** every change triggers automated regression on the critical paths (booking, token, fee, prescription, reminder) and contract tests against the health ID sandbox. The 40-hour manual regression run is replaced by an automated run of a few minutes.
- **Tester checks:** exploratory testing of changed stories, concurrency and load tests in the nightly run, and privacy checks for any change touching health records.
- **Release gate and safe rollout:** exit criteria are read from pipeline results; release goes first to a small group and can be rolled back; the SMS and health ID interfaces are monitored with alerts.
- **Retrospective:** a repository review every cycle to tune checklists and tests.

The tester's revised role

The tester becomes a quality coach and test engineer inside the team: analysing the risk of each story, building and maintaining the automated regression and contract tests, doing exploratory and privacy testing, owning the release-gate evidence and repository reports, and acting as contact with the agency for interface problems. The tester is no longer only a final checker.

Principles that gain importance

- **Early testing and planning:** two weeks leave no time for late correction.
- **Repeatable, reusable tests (regression):** each release must re-prove existing behaviour, so automation is essential.
- **Testing depends on context:** health data and money demand stricter testing than the original manual approach gave.
- **Defect clustering:** interface and shared-data areas are where most new defects will appear.
- **Test invalid and unexpected inputs at interfaces:** wrong, late or missing replies from the agency are normal cases that must be tested.

Classifying and tracking defects from the external system

Table 26: Added classification for external-system defects

Item	How it is handled
Class	New "External interface" class with sub-classes: contract/version, data mapping, availability/performance, authentication/consent, environment difference
Owner	Internal (our adapter), agency (their service) or joint (contract unclear); decides who fixes it
Severity	Based on impact on patients and data; any wrong record or privacy exposure is Critical
Tracking	Our defect ID is linked to the agency's ticket; service-level times are logged; any workaround (queue, retry, manual process) is recorded
Measures	Time to detect, time to restore, number of such defects per release, share caused by agency changes
Prevention	Contract tests on each build; production monitoring; a named contact and a shared release calendar with the agency

D. Evidence-Based Defence

CO1 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

Ans:

Defending the Final Decision

Table 27: Evidence supporting the decision to modify the plan

Evidence	What it shows	Effect on my decision
Defect data and cost	Patients found 60% of defects, which carried 91.9% of the cost (₹24,00,000 of ₹26,10,500); a production defect costs 80 times a requirement defect	Early detection still pays, so I do not discard the plan
Origins and classes	50% of incidents and 52% of defects arise in requirements and design; coding gives only 20% of incidents	Keep requirement and design checks, but small and per story
Release frequency	26 releases a year; the original plan needs about 184 person-hours per cycle against about 140 available (31% over)	Keeping it unchanged would fail; smaller reviews and automation are needed
Patient safety and privacy	Shared records turn I6-type defects into cross-hospital breaches (Critical)	Add design check, privacy testing and a strict release gate
External agency	A new defect origin beyond our control (contract, availability, mapping)	Add contract tests, monitoring and a new repository class
Team size and capacity	Five developers (about 6,400 hours over eight months) plus one tester	Changes use about 3% of developer time plus the tester; a full redesign with a bigger test team is unaffordable

Reasoning. If I kept the plan unchanged, manual review and test effort would exceed capacity by 31%, and the team would either drop activities (going back to the past, when 60 defects escaped) or delay every release. If I redesigned everything, I would throw away the parts the case data strongly support, at a cost five people cannot carry. The modified plan keeps the proven elements, shrinks them to fit two weeks and automates the repeated checks.

Estimated benefit. With the effect assumed in Task 11 (about 20 defects reaching patients instead of 60, for the same amount of work), the expected saving was about ₹9 lakh per old-sized release. The same improvement is reachable at the new pace, and the release gate, monitoring and rollback limit the damage from any escaping defect to two weeks of exposure instead of six months.

Risks I accept and how I will watch them. The automated suite takes time to build, so early cycles will have lower coverage and the tester should start with the critical paths; review and test efficiencies are estimates, and the repository will show real figures after three cycles; and the agency's behaviour is outside the team's control, so contract tests and monitoring give early warning. If the repository shows that agency-interface defects dominate and the team cannot control them, I will revisit the decision and consider redesigning the process around a dedicated integration-test group and a formal agreement with the agency.

MARK DISTRIBUTION

Task	Assessed area	Marks
Task 1	Evaluate Testing as an Engineering Activity	5
Task 2	Evaluate the Role of Process in Software Quality	2
Task 3	Evaluate Testing as a Process	6
Task 4	Apply and Critique Basic Definitions	4
Task 5	Evaluate the Application of Testing Principles	3
Task 6	Evaluate the Tester's Role	3
Task 7	Trace the Origins of Defects	4
Task 8	Evaluate Defect Classes and Severity	4
Task 9	Design and Appraise a Defect Repository	3
Task 10	Evaluate Defect Examples and Prevention Strategies	4
Task 11	Recommend and Defend a Quality Improvement Plan	4
Task 12	Day-4 Design Challenge (A–D, 2 marks each)	8
Total		**50**